3η ΑΣΚΗΣΗ ΣΤΗΝ ΑΡΧΙΤΕΚΤΟΝΙΚΗ ΥΠΟΛΟΓΙΣΤΩΝ

Όνομα: Δημήτριος

Επώνυμο: Μιχαλακόπουλος

ΑΜ: 03120229

ΜΕΡΟΣ Α

Από την εκφώνηση έχουμε τα εξής στοιχεία:

- L1 Hit Time: 1 κύκλος
- L1 Hit Rate: 94.6% ή 0.946
- L1 Miss Penalty: 49 κύκλοι
- L1 Miss Rate: 1 - 0.946 = 0.054 (ή 5.4%)
- Ποσοστό Conflict Misses: 37.5% των συνολικών misses
- Χρόνος Πρόσβασης Victim Cache: 1 κύκλος
- Στόχος Επιτάχυνσης: ≥ 1.25

Ο μέσος χρόνος πρόσβασης με ένα επίπεδο κρυφής μνήμης είναι:

$$AMAT_{L_1} = Hit\ Time_{L_1} + Miss\ Rate_{L_1} \times Miss\ Penalty_{L_1} = 1 + 0.054 \cdot 49 = 3.646$$

Ο μέσος χρόνος πρόσβασης με την προσθήκη της victim cache είναι:

$$AMAT_{L_1+VC} = Hit\ Time_{L_1} + Miss\ Rate_{L_1} \times \left(Hit\ Time_{VC} + Miss\ Rate_{VC} \times Miss\ Penalty_{L_1}\right)$$

$$= 1 + 0.054 \cdot (1 + Miss\ Rate_{VC} \cdot 49) = 1.054 + 2.646 \cdot Miss\ Rate_{VC}$$

Για να επιτύχουμε επιτάχυνση του μέσου χρόνου πρόσβασης στην μνήμη, μετά την προσθήκη της victim cache, τουλάχιστον ίση με 1.25 θα πρέπει:

$$1.25 \leq \frac{AMAT_{L_1}}{AMAT_{L_1+VC}} \Rightarrow 1.054 + 2.646 \cdot Miss\ Rate_{VC} \leq \frac{3.646}{1.25} \Rightarrow Miss\ Rate_{VC} \leq 0.704$$

Άρα, για να επιτύχουμε το ζητούμενο speedup το ποσοστό αστοχίας της victim cache πρέπει να είναι τουλάχιστον:

$$Miss\ Rate_{VC} = 0.704$$

Επομένως, το ποσοστό ευστοχίας της victim cache πρέπει να είναι τουλάχιστον:

$$Hit\ Rate_{VC} = 1 - Miss\ Rate_{VC} = 1 - 0.704 = 0.296 \text{ ή } 29.6\%$$

Έστω x το ποσοστό των conflict misses που γίνονται hits στην Victim Cache. Το συνολικό hit rate της victim cache προκύπτει από το ποσοστό των conflict misses επί το ποσοστό επιτυχίας τους:

$$Hit\ Rate_{VC} = (Ποσοστό\ Conflict\ Misses) \times x \Rightarrow x = \frac{0.296}{0.375} \Rightarrow x = 0.789 \text{ ή } 78.9\%$$

Για να επιτευχθεί επιτάχυνση τουλάχιστον ίση με 1.25, η victim cache πρέπει να αντιμετωπίζει επιτυχώς περίπου το 78.9% των conflict misses.

ΜΕΡΟΣ Β

Ερώτημα Α

Έχουμε έναν 32 bit επεξεργαστή με direct mapped, write-allocate L1 cache μεγέθους 512B, που είναι αρχικά άδεια. Το μέγεθος του block είναι 64 bytes και η μνήμη είναι byte-addressable.

- Number of Physical Address bits = 32 bits
- Cache size = 512 bytes = 2^9 bytes
- Block size = 64 bytes = 2^6 bytes
- Block offset bits = $\log_2 2^6$ = 6 bits
- Number of Blocks = 2^9 / 2^6 = 2^3 blocks
- Block index bits = $\log_2 2^3$ = 3 bits
- Block tag bits = 32 – (index bits + offset bits) = 32 – (6 + 3) = 23 bits

Επομένως, το πεδίο tag χρειάζεται 23 bits, το πεδίο index 3 bits και το πεδίο block_offset 6 bits.

Ερώτημα Β

Αρχικά, αφού τα στοιχεία των πινάκων είναι 8 bytes το καθένα, κάθε block χωράει 8 στοιχεία, το οποίο σημαίνει ότι κάθε γραμμή του πίνακα A θα βρίσκεται σε 2 συνεχόμενα blocks της μνήμης. Επιπλέον, οι γραμμές A[i], A[i+4], A[i+8], … γίνονται map στο ίδιο block, άρα τα A[0], A[4], A[8], A[12], B[0-7], για παράδειγμα, θα είναι στο ίδιο μπλοκ λόγω συνεχόμενης δήλωσης μεταβλητών.

i = 0	Read				Write	Cache Hits/Misses				
j = 0	A[4][0]	A[0][0]	A[2][0]	B[32]	A[2][0]	m	m	m	m	m
j = 1	A[4][1]	A[0][1]	A[2][1]	B[33]	A[2][1]	m	m	h	m	m
j = 2	A[4][2]	A[0][2]	A[2][2]	B[34]	A[2][2]	m	m	h	m	m
j = 3	A[4][3]	A[0][3]	A[2][3]	B[35]	A[2][3]	m	m	h	m	m
j = 4	A[4][4]	A[0][4]	A[2][4]	B[36]	A[2][4]	m	m	h	m	m
j = 5	A[4][5]	A[0][5]	A[2][5]	B[37]	A[2][5]	m	m	h	m	m
j = 6	A[4][6]	A[0][6]	A[2][6]	B[38]	A[2][6]	m	m	h	m	m
j = 7	A[4][7]	A[0][7]	A[2][7]	B[39]	A[2][7]	m	m	h	m	m

i = 1	Read				Write	Cache Hits/Misses				
j = 0	A[5][0]	A[1][0]	A[3][0]	B[40]	A[3][0]	m	m	m	m	h
j = 1	A[5][1]	A[1][1]	A[3][1]	B[41]	A[3][1]	m	m	h	h	h
j = 2	A[5][2]	A[1][2]	A[3][2]	B[42]	A[3][2]	m	m	h	h	h
j = 3	A[5][3]	A[1][3]	A[3][3]	B[43]	A[3][3]	m	m	h	h	h
j = 4	A[5][4]	A[1][4]	A[3][4]	B[44]	A[3][4]	m	m	h	h	h
j = 5	A[5][5]	A[1][5]	A[3][5]	B[45]	A[3][5]	m	m	h	h	h
j = 6	A[5][6]	A[1][6]	A[3][6]	B[46]	A[3][6]	m	m	h	h	h
j = 7	A[5][7]	A[1][7]	A[3][7]	B[47]	A[3][7]	m	m	h	h	h

i = 2	Read				Write	Cache Hits/Misses				
j = 0	A[6][0]	A[2][0]	A[4][0]	B[48]	A[4][0]	m	m	m	m	h
j = 1	A[6][1]	A[2][1]	A[4][1]	B[49]	A[4][1]	m	m	h	h	h
j = 2	A[6][2]	A[2][2]	A[4][2]	B[50]	A[4][2]	m	m	h	h	h
j = 3	A[6][3]	A[2][3]	A[4][3]	B[51]	A[4][3]	m	m	h	h	h
j = 4	A[6][4]	A[2][4]	A[4][4]	B[52]	A[4][4]	m	m	h	h	h
j = 5	A[6][5]	A[2][5]	A[4][5]	B[53]	A[4][5]	m	m	h	h	h
j = 6	A[6][6]	A[2][6]	A[4][6]	B[54]	A[4][6]	m	m	h	h	h
j = 7	A[6][7]	A[2][7]	A[4][7]	B[55]	A[4][7]	m	m	h	h	h

i = 3	Read				Write	Cache Hits/Misses				
j = 0	A[7][0]	A[3][0]	A[5][0]	B[56]	A[5][0]	m	m	m	m	h
j = 1	A[7][1]	A[3][1]	A[5][1]	B[57]	A[5][1]	m	m	h	h	h
j = 2	A[7][2]	A[3][2]	A[5][2]	B[58]	A[5][2]	m	m	h	h	h
j = 3	A[7][3]	A[3][3]	A[5][3]	B[59]	A[5][3]	m	m	h	h	h
j = 4	A[7][4]	A[3][4]	A[5][4]	B[60]	A[5][4]	m	m	h	h	h
j = 5	A[7][5]	A[3][5]	A[5][5]	B[61]	A[5][5]	m	m	h	h	h
j = 6	A[7][6]	A[3][6]	A[5][6]	B[62]	A[5][6]	m	m	h	h	h
j = 7	A[7][7]	A[3][7]	A[5][7]	B[63]	A[5][7]	m	m	h	h	h

Παρατηρούμε ότι για την τιμή i = 0 έχουμε → 7 hits και 33 misses

Παρατηρούμε ότι για τιμές i > 0 έχουμε → 22 hits και 18 misses

Άρα, τα συνολικά αποτελέσματα (write-allocate) για την εκτέλεση του παραπάνω κώδικα είναι:

73 hits και 87 misses

Ερώτημα Γ

Σε αυτή την περίπτωση που η cache αλλάζει σε write-no-allocate, όταν έχουμε Write Miss το block δεν μεταφέρεται στην cache. Έτσι η εκτέλεση του ίδιου κώδικα θα γίνει ως εξής:

i = 0	Read				Write	Cache Hits/Misses				
j = 0	A[4][0]	A[0][0]	A[2][0]	B[32]	A[2][0]	m	m	m	m	m
j = 1	A[4][1]	A[0][1]	A[2][1]	B[33]	A[2][1]	m	m	m	m	m
j = 2	A[4][2]	A[0][2]	A[2][2]	B[34]	A[2][2]	m	m	m	m	m
j = 3	A[4][3]	A[0][3]	A[2][3]	B[35]	A[2][3]	m	m	m	m	m
j = 4	A[4][4]	A[0][4]	A[2][4]	B[36]	A[2][4]	m	m	m	m	m
j = 5	A[4][5]	A[0][5]	A[2][5]	B[37]	A[2][5]	m	m	m	m	m
j = 6	A[4][6]	A[0][6]	A[2][6]	B[38]	A[2][6]	m	m	m	m	m
j = 7	A[4][7]	A[0][7]	A[2][7]	B[39]	A[2][7]	m	m	m	m	m

i = 1	Read				Write	Cache Hits/Misses				
j = 0	A[5][0]	A[1][0]	A[3][0]	B[40]	A[3][0]	m	m	m	m	h
j = 1	A[5][1]	A[1][1]	A[3][1]	B[41]	A[3][1]	m	m	h	h	h
j = 2	A[5][2]	A[1][2]	A[3][2]	B[42]	A[3][2]	m	m	h	h	h
j = 3	A[5][3]	A[1][3]	A[3][3]	B[43]	A[3][3]	m	m	h	h	h
j = 4	A[5][4]	A[1][4]	A[3][4]	B[44]	A[3][4]	m	m	h	h	h
j = 5	A[5][5]	A[1][5]	A[3][5]	B[45]	A[3][5]	m	m	h	h	h
j = 6	A[5][6]	A[1][6]	A[3][6]	B[46]	A[3][6]	m	m	h	h	h
j = 7	A[5][7]	A[1][7]	A[3][7]	B[47]	A[3][7]	m	m	h	h	h

i = 2	Read				Write	Cache Hits/Misses				
j = 0	A[6][0]	A[2][0]	A[4][0]	B[48]	A[4][0]	m	m	m	m	h
j = 1	A[6][1]	A[2][1]	A[4][1]	B[49]	A[4][1]	m	m	h	h	h
j = 2	A[6][2]	A[2][2]	A[4][2]	B[50]	A[4][2]	m	m	h	h	h
j = 3	A[6][3]	A[2][3]	A[4][3]	B[51]	A[4][3]	m	m	h	h	h
j = 4	A[6][4]	A[2][4]	A[4][4]	B[52]	A[4][4]	m	m	h	h	h
j = 5	A[6][5]	A[2][5]	A[4][5]	B[53]	A[4][5]	m	m	h	h	h
j = 6	A[6][6]	A[2][6]	A[4][6]	B[54]	A[4][6]	m	m	h	h	h
j = 7	A[6][7]	A[2][7]	A[4][7]	B[55]	A[4][7]	m	m	h	h	h

i = 3	Read				Write	Cache Hits/Misses				
j = 0	A[7][0]	A[3][0]	A[5][0]	B[56]	A[5][0]	m	m	m	m	h
j = 1	A[7][1]	A[3][1]	A[5][1]	B[57]	A[5][1]	m	m	h	h	h
j = 2	A[7][2]	A[3][2]	A[5][2]	B[58]	A[5][2]	m	m	h	h	h
j = 3	A[7][3]	A[3][3]	A[5][3]	B[59]	A[5][3]	m	m	h	h	h
j = 4	A[7][4]	A[3][4]	A[5][4]	B[60]	A[5][4]	m	m	h	h	h
j = 5	A[7][5]	A[3][5]	A[5][5]	B[61]	A[5][5]	m	m	h	h	h
j = 6	A[7][6]	A[3][6]	A[5][6]	B[62]	A[5][6]	m	m	h	h	h
j = 7	A[7][7]	A[3][7]	A[5][7]	B[63]	A[5][7]	m	m	h	h	h

Παρατηρούμε ότι για την τιμή i = 0 έχουμε → 0 hits και 40 misses

Παρατηρούμε ότι για τιμές i > 0 έχουμε → 22 hits και 18 misses

Άρα, τα συνολικά αποτελέσματα (write-no-allocate) για την εκτέλεση του ίδιου κώδικα είναι:

66 hits και 94 misses